SCREAMING ARCHITECTURE

The 30-Minute Field Guide

From Zero to Clean Architecture in One Sitting

Everything you need to stop writing spaghetti code and start designing systems that **scream** their intent.

PromptPolish AI — 2026

Screaming Architecture: The 30-Minute Field Guide

Author: PromptPolish AI

Version: 1.0 — June 2026

Introduction

"A good architecture is one that makes the system easy to understand, easy to change, and easy to deploy." --- Robert C. Martin

Most codebases don't *scream* their intent. You open a project and see `utils/`, `helpers/`, `services/`, `managers/`. Nothing tells you what the **application does**. Is it an e-commerce platform? A banking system? A game?

Screaming Architecture fixes this. Your project structure should **scream** "I AM AN E-COMMERCE SYSTEM" at first glance.

Chapter 1: The Core Principle

Your code should tell a story before anyone reads a single line.

If someone opens your project and sees:

```
src/
  orders/
  payments/
  inventory/
  shipping/
```

They instantly know: *this handles orders, payments, inventory, and shipping*. The architecture **screams** its purpose.

Compare with:

```
src/
  utils/
  models/
  controllers/
  services/
```

What does this app do? **No idea.** That's the problem.

The Rule of First Glance

A new developer should understand your domain within 10 seconds of opening the project. If they can't, your architecture is failing.

Chapter 2: Package by Feature, Not by Layer

This is the single highest-impact change you can make.

[Package by Layer (Anti-pattern)

```
src/
  controllers/
    UserController.java
    OrderController.java
  models/
    User.java
    Order.java
  repositories/
    UserRepository.java
    OrderRepository.java
  services/
    UserService.java
    OrderService.java
```

Problems:

- Features are scattered across 4 directories
- To understand "Orders", you open 4+ directories
- Every new feature touches EVERY layer directory
- High coupling, low cohesion

[OK] Package by Feature

```
src/
  orders/
    OrderController.java
    OrderService.java
    OrderRepository.java
    OrderMapper.java
    OrderDTO.java
  users/
    UserController.java
    UserService.java
    UserRepository.java
    UserMapper.java
    UserDTO.java
```

Benefits:

- Each feature is self-contained
- To understand "Orders", open ONE directory
- Adding a feature = adding ONE directory
- Easy to extract into microservices later
- High cohesion, loose coupling

Chapter 3: The Screaming Structure

Here's the template I use for every project, regardless of framework:

```
src/
  <domain>/
    application/
      <use-cases>.ts          # What the app CAN DO
      <ports>.ts              # Interfaces (contracts)
    domain/
      <entities>.ts           # Core business objects
      <value-objects>.ts      # Immutable value types
      <events>.ts             # Domain events
    infrastructure/
      <repositories>.ts       # Database implementations
      <external-apis>.ts      # Third-party integrations
      <mappers>.ts            # Data transformation
    presentation/
      <controllers>.ts        # HTTP/API layer
      <validators>.ts         # Input validation
      <serializers>.ts        # Response formatting
```

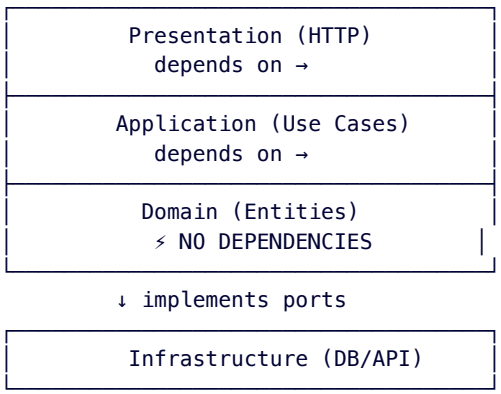
Real Example: Banking App

```
src/
  accounts/
    application/
      OpenAccountUseCase.ts
      TransferMoneyUseCase.ts
      CloseAccountUseCase.ts
    domain/
      Account.ts
      AccountNumber.ts (value object)
      Money.ts (value object)
      AccountOpenedEvent.ts
    infrastructure/
      PostgresAccountRepository.ts
      BankTransferApi.ts
    presentation/
      AccountController.ts
      CreateAccountValidator.ts
  loans/
    application/
      ApplyForLoanUseCase.ts
      ApproveLoanUseCase.ts
    domain/
      Loan.ts
      InterestRate.ts (value object)
      LoanApplication.ts
    infrastructure/
      PostgresLoanRepository.ts
      CreditScoreApi.ts
    presentation/
      LoanController.ts
      LoanApplicationValidator.ts
```

This structure **screams**: "I AM A BANKING APPLICATION."

Chapter 4: The Dependency Rule

Dependencies point **INWARD**. Domain knows nothing. Application depends on domain. Infrastructure implements application ports. Presentation depends on application.



The Golden Rule

> *"Source code dependencies always point INWARD, toward the domain."*

Your domain layer should have ZERO imports from frameworks. ZERO from databases. ZERO from HTTP libraries. If your `Account.ts` imports Express, Django, or Hibernate, **you're doing it wrong**.

Chapter 5: Use Cases as First-Class Citizens

Your application's **behavior** should be explicit. Each use case is a class with a single public method.

```
// This screams: "I transfer money between accounts"
class TransferMoneyUseCase {
  constructor(
    private accountRepo: AccountRepository,
    private eventBus: EventBus
  ) {}

  execute(command: TransferCommand): Result {
    const from = this.accountRepo.findById(command.fromAccountId)
    const to = this.accountRepo.findById(command.toAccountId)

    from.withdraw(command.amount)
    to.deposit(command.amount)

    this.accountRepo.save(from)
    this.accountRepo.save(to)
    this.eventBus.publish(new MoneyTransferredEvent(/*...*/))

    return Result.success()
  }
}
```

Naming convention: `UseCase`

`PlaceOrderUseCase`, `CancelSubscriptionUseCase`, `ApproveRefundUseCase`. Each name screams **what the system does**.

Chapter 6: Testing That Screams, Too

Your test structure should mirror your source structure EXACTLY.

```
tests/
  accounts/
    application/
      OpenAccountUseCase.test.ts
      TransferMoneyUseCase.test.ts
    domain/
      Account.test.ts
      Money.test.ts (value object)
    infrastructure/
      PostgresAccountRepository.test.ts
    presentation/
      AccountController.test.ts
```

Test naming: `Should_When`

```
test("ShouldTransferFunds_WhenSufficientBalance")
test("ShouldRejectTransfer_WhenInsufficientBalance")
test("ShouldNotifyBeneficiary_WhenTransferCompletes")
```

When tests fail, the failure message screams exactly what broke.

Chapter 7: Framework Independence (The Litmus Test)

Here's how you know you're doing it right: **Could you swap your framework without changing business logic?**

If you use Express today, could you migrate to Fastify or NestJS by only changing the `presentation/` layer? If not, you're coupled.

Framework Coupling Sins

```
// [ SIN: Domain depends on framework
// account/domain/Account.ts
import { ObjectId } from 'mongoose'  // WHY is Mongoose in the domain?!

export class Account {
  constructor(
    public id: ObjectId,  // Now you can NEVER leave Mongoose
    public balance: number
  ) {}
}

// [OK] VIRTUE: Domain is pure
// account/domain/Account.ts
```

```
export class Account {
  constructor(
    public id: string,    // Plain string. Any DB works.
    public balance: number
  ) {}
}
```

Your domain should compile in a **text editor**. No framework required. If your domain file has a single import from a framework, **refactor it NOW**.

Chapter 8: Real Talk --- The Objections

"This is overengineering for a small project."

No. This scales DOWN. A Todo app with Screaming Architecture:

```
src/
  todos/
    application/
      CreateTodoUseCase.ts
      CompleteTodoUseCase.ts
    domain/
      Todo.ts
      TodoId.ts
    infrastructure/
      InMemoryTodoRepository.ts
    presentation/
      TodoController.ts
```

6 files. Zero overengineering. Maximum clarity.

"My framework doesn't support this."

Your framework serves YOU. Not the other way around. Frameworks are tools, not masters. If your framework forces a specific structure, **create a** `src/` **layer on top** that follows screaming principles, and let the framework layer be infrastructure.

"It takes too long."

Question: What's more expensive — 30 extra minutes structuring your project now, or 30 hours debugging unstructured code 6 months from now?

Chapter 9: The 10-Minute Audit

Grab any project you've worked on. Score each question 0 or 1:

Can you tell what the app does by looking at `src/`? (__)

Are features in their own directories? (__)

Does domain have ZERO framework imports? (__)

Are use cases named as ? (__)

Does infrastructure implement interfaces from application? (__)

Would swapping the framework require only presentation changes? (__)

Does test structure mirror source structure? (__)

Score 7/7: You're a Screaming Architect. Teach others.

Score 4-6: Good foundation. Fix the gaps this week.

Score 0-3: Start applying Chapter 2 and 3 TODAY.

Chapter 10: Beyond Code --- Architecture as Culture

Screaming Architecture isn't just folder structure. It's a **mindset**:

- Your **README** should scream the purpose
- Your **PR titles** should scream the change
- Your **commit messages** should scream the intent
- Your **team's vocabulary** should scream the domain

When every artifact of your development process screams intent, you stop writing code that *happens to work* and start **designing systems that endure**.

Conclusion

Great architecture isn't about patterns, frameworks, or buzzwords. It's about **communication**. Your codebase communicates with every developer who touches it — present and future.

Make it scream.

About the Author

PromptPolish AI specializes in transforming chaotic codebases into screaming architectures. With 15+ years of experience across startups and enterprise, we've seen code that makes us cry and code that makes us proud.

This guide condenses hundreds of architecture reviews into actionable patterns you can apply TODAY.

Start with one module. Make it scream. The rest will follow.
